```lua
local MovementMain = require(game.ServerScriptService.Modules.MovementMain)
local SFXMain = require(game.ReplicatedStorage.Modules.SFXMain)
local VFXMain = require(game.ReplicatedStorage.Modules.VFXMain)
local AdminSystem = require(game.ReplicatedStorage.Modules.AdminSystem)
local CombatMain = require(game.ServerScriptService.Modules.CombatMain)
local DestructionMain = require(game.ReplicatedStorage.Modules.DestroyableMain)
local MoveMain = require(game.ReplicatedStorage.Modules.MoveMain)
local DataMain = require(game.ReplicatedStorage.Modules.DataMain)
local EmoteMain = require(game.ReplicatedStorage.Modules.EmoteMain)
local Data = game:GetService("DataStoreService"):GetDataStore("Player", "V1")

game.Workspace.VFXStuff:Destroy()
local function Respawn(Player)
    if Player == nil then return end
    Player:LoadCharacter()
end

local RegisteredHumanoids = {}
function RegisterHumanoid(Humanoid:Humanoid)
    if table.find(RegisteredHumanoids, Humanoid) then return end
    table.insert(RegisteredHumanoids, Humanoid)
    for Name, Value in pairs(script.BaseHuman:GetAttributes()) do
        Humanoid:SetAttribute(Name, Value)
    end
    for a,b in pairs(script.BaseHuman:GetChildren()) do
        b:Clone().Parent = Humanoid
    end
    Humanoid.RequiresNeck = false

    local Animations:{AnimationClip} = {}
    local Events:{RBXScriptConnection} = {}
    local Dead = false
    local Stun = 0
    local StunVel = Instance.new("LinearVelocity")
    StunVel.Name = "StunVel"
    StunVel.ForceLimitMode = Enum.ForceLimitMode.PerAxis
    StunVel.MaxAxesForce = Vector3.new(999999, 0, 999999)
    StunVel.VectorVelocity = Vector3.new(0, 0, 0)
    local StunvelAttach = Instance.new("Attachment", Humanoid.Parent.HumanoidRootPart)
    StunVel.Attachment0 = StunvelAttach
    StunvelAttach.Name = "VelAttach"
    local Event = Instance.new("BindableEvent", Humanoid)
    Event.Name = "Event"
    local Run = Instance.new("RemoteEvent", Humanoid)
    Run.Name = "Run"
    local Inputs = Instance.new("RemoteEvent", Humanoid)
    Inputs.Name = "Inputs"
    local Moves = Instance.new("Folder", Humanoid)
    Moves.Name = "Moves"
    local Disable = 0
    local MovementModifiers:{{Time:number, SpeedMultiplier:number, JumpMultiplier:number, Name:string}} = {}
    local SuperArmorDuration = 0
    local KnockbackVel = 0
    local KnockbackDuration = 0
    local RagdollDuration = 0
    local IFrames = 0
    local LocalIFrames = {}
    local RagdollRemove:{Instance} = {}
    local RagdollFix:{Instance} = {}
    local HumanoidRootPart:Part = Humanoid.Parent.HumanoidRootPart
    local Bounds = false
    local CanRespawn = true
    local Player = game.Players:GetPlayerFromCharacter(Humanoid.Parent)
    local LastPos = Vector3.new()
    local GetIFrame = Instance.new("BindableFunction", Humanoid)
    GetIFrame.Name = "GetIFrames"
```

```lua

    for a,b in pairs(Humanoid.Parent:GetDescendants()) do
        if b:IsA("BasePart") then
            b.CollisionGroup = "Player"
        end
    end

    local Ease:Tween = nil
    local Connector = {
        ["Animation"] = function(Animation:string)
            if Animations[Animation] then
                Animations[Animation]:Play()
            end
        end,
        ["Flinch"] = function(Priority:number)
            if Priority == nil then Priority = 1 end
            if Priority >= 2 then
                if math.random(1, 2) == 1 then
                    Animations.HitRight:Play()
                else
                    Animations.HitLeft:Play()
                end
            end
        end,
        ["Movement"] = function(Enable:boolean, Data)
            if Enable then
                MovementModifiers[Data.Name] = {
                    ["Time"] = Data.Time,
                    ["SpeedMultiplier"] = Data.SpeedMultiplier,
                    ["JumpMultiplier"] = Data.JumpMultiplier
                }
            else
                MovementModifiers[Data.Name] = nil
            end
        end,
        ["Stun"] = function(Duration:number)
            if Stun < Duration then
                if Stun <= 0 then
                    Humanoid:SetAttribute("Stunned", true)
                    if Humanoid.Parent.HumanoidRootPart:FindFirstChildOfClass("BodyVelocity") and Humanoid.Parent.HumanoidRootPart:FindFirstChildOfClass("BodyVelocity") ~= StunVel then
                        Humanoid.Parent.HumanoidRootPart:FindFirstChildOfClass("BodyVelocity"):Destroy()
                    end
                    if StunVel.Parent == nil then
                        StunVel.Parent = Humanoid.Parent.HumanoidRootPart
                    end
                end
                Stun = Duration
            end
        end,
        ["Disable"] = function(Duration:number)
            if Disable < Duration then
                if Disable <= 0 then
                    Humanoid:SetAttribute("Disabled", true)
                end
                Disable = Duration
            end
        end,
        ["IFrames"] = function(Duration:number, Local:Model)
            Local = nil
            if Local then
                LocalIFrames[Local] = Duration
            end
            if IFrames < Duration then
```

```
                if IFrames <= 0 then
                    Humanoid:SetAttribute("IFrames", true)
                end
                IFrames = Duration
            end
        end,
        ["Ragdoll"] = function(Duration)
            if Duration == false then
                RagdollDuration = -0.1
                Humanoid:SetAttribute("Ragdolled", false)
                for a,b in pairs(RagdollFix) do
                    b.Enabled = true
                end

                for a,b in pairs(RagdollRemove) do
                    b:Destroy()
                end

                if game.Players:GetPlayerFromCharacter(Humanoid.Parent) then
                    game.ReplicatedStorage.Remotes.Ragdoll:FireClient(game.Players:GetPlayerFromCharacter(Humanoid.Parent), false)
                else
                    Humanoid:ChangeState(Enum.HumanoidStateType.Running)
                end
                return
            end
            if RagdollDuration <= 0 then
                RagdollDuration = Duration
                Humanoid:SetAttribute("Ragdolled", true)
                Stun = 0
                Humanoid:SetAttribute("Stunned", false)

                RagdollRemove = {}
                RagdollFix = {}

                if game.Players:GetPlayerFromCharacter(Humanoid.Parent) then
                    game.ReplicatedStorage.Remotes.Ragdoll:FireClient(game.Players:GetPlayerFromCharacter(Humanoid.Parent), true)
                else
                    Humanoid:ChangeState(Enum.HumanoidStateType.Physics)
                end

                for index,joint in pairs(Humanoid.Parent:GetDescendants()) do
                    if joint.Parent.Parent == Humanoid.Parent then
                        if joint:IsA("Motor6D") and joint.Name ~= "RootJoint" then
                            local socket = Instance.new("BallSocketConstraint")
                            local Collide = Instance.new("NoCollisionConstraint", socket)
                            local a1 = Instance.new("Attachment")
                            a1.Name = "RagdollAttachment"
                            local a2 = Instance.new("Attachment")
                            a2.Name = "RagdollAttachment"
                            a1.Parent = joint.Part0
                            a2.Parent = joint.Part1
                            socket.Name = "RagdollJoint"
                            socket.Parent = joint.Parent
                            socket.Attachment0 = a1
                            socket.Attachment1 = a2
                            a1.CFrame = joint.C0 + (joint.C0.Position * 0.1)
                            a2.CFrame = joint.C1
                            socket.LimitsEnabled = true
                            socket.TwistLimitsEnabled = true
                            joint.Enabled = false
                            Collide.Part0 = joint.Part0
                            Collide.Part1 = joint.Part1
                            table.insert(RagdollFix, joint)
                            table.insert(RagdollRemove, socket)
                            table.insert(RagdollRemove, Collide)
```

```
                                table.insert(RagdollRemove, a1)
                                table.insert(RagdollRemove, a2)
                            end
                        end
                    end
                else
                    if Duration > RagdollDuration then
                        RagdollDuration = Duration
                    end
                end
            end,
            ["SuperArmor"] = function(Duration)
                Humanoid:SetAttribute("SuperArmor", true)
                SuperArmorDuration = Duration
                if Duration == false then
                    SuperArmorDuration = 0
                    Humanoid:SetAttribute("SuperArmor", false)
                end
            end,
            ["Knockback"] = function(Knockback, Duration, MaxForce, Extra:{})
                KnockbackDuration = Duration
                if Duration == 0 then KnockbackDuration = 0 return end
                if Humanoid.Parent.HumanoidRootPart:FindFirstChild("DashVel") then
                    Humanoid.Parent.HumanoidRootPart:FindFirstChild("DashVel"):Destroy()
                end
                if Ease ~= nil then
                    Ease:Cancel()
                end
                if StunVel.Parent ~= nil then
                    StunVel.VectorVelocity = Knockback
                    if Extra and Extra.Ease then
                        Ease = game:GetService("TweenService"):Create(StunVel, TweenInfo.new(Extra.EaseTime, Enum.EasingStyle.Quad, Enum.EasingDirection.Out), {VectorVelocity = Extra.Ease})
                        Ease:Play()
                    end
                    if MaxForce ~= nil then
                        StunVel.MaxAxesForce = MaxForce
                    end
                else
                    StunVel.Parent = Humanoid.Parent.HumanoidRootPart
                    StunVel.VectorVelocity = Knockback
                    if Extra and Extra.Ease then
                        Ease = game:GetService("TweenService"):Create(StunVel, TweenInfo.new(Extra.EaseTime, Enum.EasingStyle.Quad, Enum.EasingDirection.Out), {VectorVelocity = Extra.Ease})
                        Ease:Play()
                    end
                    if MaxForce ~= nil then
                        StunVel.MaxAxesForce = MaxForce
                    end
                end
            end
        }

        local function FireEvent(Type, ...)
            local Args = {...}
            if Connector[Type] then
                Connector[Type](table.unpack(Args))
            end
        end

        table.insert(Events, Event.Event:Connect(FireEvent))

        table.insert(Events, Humanoid.Parent.ChildAdded:Connect(function(b)
            if b:IsA("Accessory") then
                pcall(function()
```

```
                b:FindFirstChildOfClass("Part").CanQuery = false
            end)
        end
    end))
    for a,b in pairs(Humanoid.Parent:GetChildren()) do
        if b:IsA("Accessory") then
            pcall(function()
                b:FindFirstChildOfClass("Part").CanQuery = false
            end)
        end
    end

    table.insert(Events, StunVel.Destroying:Connect(function()
        Humanoid.Health = 0
    end))

    table.insert(Events, Humanoid.Died:Connect(function()
        FireEvent("Ragdoll", 1000)
        wait(3)
        Dead = true
        repeat
            wait()
        until CanRespawn == true
        if Player then
            Respawn(Player)
        end
    end))

    local Fall = 0
    local LastJump = 0
    local JumpHeight = 50
    table.insert(Events, Humanoid:GetPropertyChangedSignal("Jump"):Connect(function()
        if Humanoid.JumpPower < 20 then return end
        if Humanoid.Jump == false then return end
        LastJump = game.Workspace:GetServerTimeNow()
        JumpHeight = math.clamp(JumpHeight * 0.98, 25, 50)
    end))

    if not Humanoid:FindFirstChild("Animator") then
        Instance.new("Animator", Humanoid)
    end
    for a,b in pairs(script.Animations:GetChildren()) do
        Animations[b.Name] = Humanoid.Animator:LoadAnimation(b)
    end

    local Running = false
    table.insert(Events, Run.OnServerEvent:Connect(function(Event, Run)
        if Event ~= Player then AdminSystem.Ban(Player, "invalid remote event sender",
        math.huge) end
        Running = Run
    end))

    local function StartRun()
        Humanoid:SetAttribute("Running", true)
    end

    local function StopRun()
        Humanoid:SetAttribute("Running", false)
    end

    GetIFrame.OnInvoke = function(Target)
        for a,b in pairs(LocalIFrames) do
            if Target == a then return true end
        end
        return nil
    end

```

```lua
    while not Dead do
        -- [ Statuses ] --
        local Delta = task.wait(1/60)
        if Humanoid.Parent == nil then break end
        if Humanoid.Parent.Parent == nil then break end
        if HumanoidRootPart.Parent == nil then break end
        if IFrames > 0 then
            IFrames -= Delta

            if IFrames <= 0 then
                Humanoid:SetAttribute("IFrames", false)
            end
        end
        for a,b in pairs(LocalIFrames) do
            LocalIFrames[a] -= Delta
            if LocalIFrames[a] <= 0 then LocalIFrames[a] = nil end
        end
        if Stun > 0 then
            Stun -= Delta

            if Stun <= 0 and KnockbackDuration <= 0 then
                Humanoid:SetAttribute("Stunned", false)
                StunVel.Parent = nil
            end
        elseif Humanoid:GetAttribute("Stunned") and Stun < 0 then
            Humanoid:SetAttribute("Stunned", false)
            StunVel.Parent = nil
        end
        if Disable > 0 then
            Disable -= Delta

            if Disable <= 0 then
                Humanoid:SetAttribute("Disabled", false)
            end
        end
        if KnockbackDuration > 0 then
            KnockbackDuration -= Delta

            if KnockbackDuration <= 0 then
                if Stun <= 0 then
                    StunVel.Parent = nil
                    StunVel.VectorVelocity = Vector3.new()
                    StunVel.MaxAxesForce = Vector3.new(999999, 0, 999999)
                else
                    StunVel.VectorVelocity = Vector3.new()
                    StunVel.MaxAxesForce = Vector3.new(999999, 0, 999999)
                end
            end
        end
        if RagdollDuration > 0 then
            if HumanoidRootPart.AssemblyLinearVelocity.Magnitude > 5 and (LastPos - HumanoidRootPart.Position).Magnitude > 4 then
                local Size = HumanoidRootPart.AssemblyLinearVelocity.Magnitude/10
                local Pos = CFrame.new(HumanoidRootPart.Position, HumanoidRootPart.Position + HumanoidRootPart.AssemblyLinearVelocity)
                Pos += Pos.LookVector * (Size/2)
                DestructionMain.ForceHitbox({
                    Position = Pos,
                    Size = Vector3.new(5, 5, math.clamp(Size, 5, 100)),
                    Knockback = HumanoidRootPart.AssemblyLinearVelocity,
                    Player = Player
                })
                LastPos = HumanoidRootPart.Position
            end
            local Params = RaycastParams.new()
            Params.FilterDescendantsInstances = {game.Workspace.Ignore, script.Parent}
            if game.Workspace:Raycast(HumanoidRootPart.Position, Vector3.new(0, -3, 0),
```

```
            Params) then
                RagdollDuration -= Delta
            elseif RagdollDuration < 1 then RagdollDuration = 1
            end

            if RagdollDuration <= 0 then
                Humanoid:SetAttribute("Ragdolled", false)
                for a,b in pairs(RagdollFix) do
                    b.Enabled = true
                end

                for a,b in pairs(RagdollRemove) do
                    b:Destroy()
                end

                if game.Players:GetPlayerFromCharacter(Humanoid.Parent) then
                    game.ReplicatedStorage.Remotes.Ragdoll:FireClient(
                    game.Players:GetPlayerFromCharacter(Humanoid.Parent), false)
                else
                    Humanoid:ChangeState(Enum.HumanoidStateType.GettingUp)
                end
                FireEvent("SuperArmor", 0.5)
            end
        end
        if SuperArmorDuration > 0 then
            SuperArmorDuration -= Delta

            if SuperArmorDuration <= 0 then
                Humanoid:SetAttribute("SuperArmor", false)
            end
        end

        -- [ Movement ] --

        local Direction = HumanoidRootPart.CFrame:VectorToObjectSpace(
        Humanoid.MoveDirection)
        if (Humanoid:GetAttribute("Running") and CombatMain.CanAttack(Humanoid)) or not
        Running then
            StopRun()
        elseif Running and not Humanoid:GetAttribute("Running") and not
        CombatMain.CanAttack(Humanoid) then
            StartRun()
        end

        if LastJump ~= 0 and game.Workspace:GetServerTimeNow() - LastJump > 1 then
            LastJump = 0
            JumpHeight = 50
        end
        local WalkSpeed = 16
        local JumpPower = JumpHeight
        local ModifierLength = 0
        for a,b in pairs(MovementModifiers) do
            ModifierLength += 1
        end
        for a,b in pairs(MovementModifiers) do
            b.Time -= Delta
            if b.Time <= 0 then MovementModifiers[a] = nil continue end
            if a == "Run" and ModifierLength > 1 then continue end
            WalkSpeed *= b.SpeedMultiplier
            JumpPower *= b.JumpMultiplier
        end
        if Humanoid:GetAttribute("Blocking") then
            WalkSpeed *= 0.25
            JumpPower = 0
        end
        if Humanoid:GetAttribute("Active") or Humanoid:GetAttribute("Stunned") then
            JumpPower = 0
```

```
        end

        Humanoid:SetAttribute("WalkSpeed", WalkSpeed)
        Humanoid.JumpPower = JumpPower

        --[[if not Bounds then
            if HumanoidRootPart.Position.X < game.Workspace.Walls.PlayArea.Position.X - game.Workspace.Walls.PlayArea.Size.X/2
                or HumanoidRootPart.Position.X > game.Workspace.Walls.PlayArea.Position.X + game.Workspace.Walls.PlayArea.Size.X/2
                or HumanoidRootPart.Position.Z < game.Workspace.Walls.PlayArea.Position.Z - game.Workspace.Walls.PlayArea.Size.Z/2
                or HumanoidRootPart.Position.Z > game.Workspace.Walls.PlayArea.Position.Z + game.Workspace.Walls.PlayArea.Size.Z/2
                or HumanoidRootPart.Position.Y < game.Workspace.Walls.PlayArea.Position.Y - game.Workspace.Walls.PlayArea.Size.Y/2
                or HumanoidRootPart.Position.Y > game.Workspace.Walls.PlayArea.Position.Y + game.Workspace.Walls.PlayArea.Size.Y/2 then
                Bounds = true
            end
            if Bounds then
                coroutine.wrap(function()
                    CanRespawn = false
                    SFXMain.PlaySound({
                        Part = HumanoidRootPart,
                        Sound = game.ReplicatedStorage.SFX.BellToll
                    })
                    HumanoidRootPart.Anchored = true
                    local WhiteHand = game.ServerStorage.Extras.WhiteHand:Clone()
                    WhiteHand.Position = HumanoidRootPart.Position + Vector3.new(0, 200, 0)
                    WhiteHand.Parent = game.Workspace.Ignore.VFX
                    game:GetService("TweenService"):Create(WhiteHand, TweenInfo.new(2, Enum.EasingStyle.Cubic, Enum.EasingDirection.Out), {Position = HumanoidRootPart.Position + Vector3.new(0, 5, 0)}):Play()
                    wait(2)
                    if Humanoid.Parent ~= nil and Humanoid.Health > 30 then
                        local SpawnPoint:SpawnLocation = game.Workspace.Spawns:GetChildren()[math.random(1, #game.Workspace.Spawns:GetChildren())]
                        HumanoidRootPart.CFrame = CFrame.new(SpawnPoint.Position + Vector3.new(0, 50, 0))
                        HumanoidRootPart.Anchored = false
                        WhiteHand.Position = HumanoidRootPart.Position + Vector3.new(0, 5, 0)
                        Humanoid:TakeDamage(30)
                        Bounds = false
                        CanRespawn = true
                    else
                        SFXMain.PlaySound({
                            Part = WhiteHand,
                            Sound = game.ReplicatedStorage.SFX.yummers
                        })
                        wait(1)
                        if Humanoid.Parent ~= nil then
                            for a,b in pairs(Humanoid.Parent:GetDescendants()) do
                                if b:IsA("BasePart") then b.Transparency = 1 end
                            end
                            Humanoid.Health = -100
                        end
                        wait(0.5)
                        CanRespawn = true
```

```
                    end
                    game:GetService("TweenService"):Create(WhiteHand, TweenInfo.new(2, Enum.EasingStyle.Cubic, Enum.EasingDirection.In), {Position = WhiteHand.Position + Vector3.new(0, 200, 0)}):Play()
                    game:GetService("Debris"):AddItem(WhiteHand, 2)
                end)()
            end
        end]]
    end
    for a,b in pairs(Events) do b:Disconnect() end
    table.remove(RegisteredHumanoids, table.find(RegisteredHumanoids, Humanoid))
    if HumanoidRootPart.Parent == nil then Respawn(Player) end
end

script.GetHumanoid.OnInvoke = function(Humanoid)
    return RegisteredHumanoids[Humanoid]
end

_G.PlayerData = {}
function PlayerAdded(Player:Player)
    local PlayerData
    local Success, Err
    local Count = 0
    local Animations = {}
    repeat
        Success, Err = pcall(function()
            PlayerData = Data:GetAsync(Player.UserId)
        end)
        Count += 1
        if not Success then
            if Count >= 5 then
                Player:Kick("Failed to load data: " .. Err)
            end
            wait(5)
        end
    until Success

    if PlayerData == nil then
        PlayerData = {
            ["Character"] = "Dragon of Dojima",
            ["Kills"] = 0,
            ["Money"] = 0,
            ["Skins"] = {},
            ["EquippedSkins"] = {["Dragon of Dojima"] = nil},
        }
    end
    if PlayerData.EquippedSkins == nil then
        PlayerData.EquippedSkins = {}
    end
    if PlayerData.EquippedEmotes == nil then
        PlayerData.EquippedEmotes = {}
    end
    if PlayerData.EquippedSkins["Dragon of Dojima"] == nil then
        PlayerData.EquippedSkins["Dragon of Dojima"] = nil
    end
    if PlayerData.Emotes == nil then
        PlayerData.Emotes = {}
        for a,b in pairs(EmoteMain.Emotes) do
            if PlayerData.Emotes[a] == nil then
                PlayerData.Emotes[a] = {}
            end
        end
    end
    _G.PlayerData[Player.UserId] = PlayerData
    print("Loaded Data.")

    local Leaderstats = Instance.new("Folder", Player)
```

```
    Leaderstats.Name = "leaderstats"

    local Kills = Instance.new("NumberValue")
    Kills.Value = DataMain.GetData(Player).Kills
    Kills.Name = "Kills"
    Kills.Parent = Leaderstats

    local Data = Instance.new("Folder", Player)
    Data.Name = "Data"

    local EquippedEmotes = Instance.new("Folder", Data)
    EquippedEmotes.Name = "EquippedEmotes"

    Player:SetAttribute("Ultimate", 0)
    Player:SetAttribute("MaxUltimate", 1)

    for a = 1, 8 do
        local Emote = Instance.new("StringValue", EquippedEmotes)
        Emote.Name = a
        Emote.Value = "Griddy"
    end

    local CharacterSelected = Instance.new("StringValue", Data)
    CharacterSelected.Name = "Character"
    CharacterSelected.Value = "Dedicated Lawyer"

    local Skin = Instance.new("StringValue", Data)
    Skin.Name = "Skin"
    Skin.Value = DataMain.GetData(Player).EquippedSkins[CharacterSelected.Value] or ""

    local LivingData

    for Name, Value in pairs(script.BasePlayer:GetAttributes()) do
        Player:SetAttribute(Name, Value)
    end

    local Events = {}
    local Emoting = false
    Player.CharacterAdded:Connect(function(Character)
        Emoting = false
        for a,b in pairs(Events) do
            b:Disconnect()
        end
        local function Emote(Animation:string)
            if Emoting then return end
            if CombatMain.CanAttack(Character.Humanoid) then return end
            local EmoteEvents = {}
            local PhysicalAnimation = game.ReplicatedStorage.Animations.Emotes[Animation]
            if PhysicalAnimation:GetAttribute("WalkSpeed") then
                MovementMain.ChangeMovement(true, Character.Humanoid, {
                    SpeedMultiplier = PhysicalAnimation:GetAttribute("WalkSpeed"),
                    JumpMultiplier = 1,
                    Name = "Emote",
                    Time = math.huge
                })
            end
            local function StopEmote()
                Emoting = false
                for a,b in pairs(EmoteEvents) do
                    b:Disconnect()
                end
                Animations[Animation]:Stop()
                MovementMain.ChangeMovement(false, Character.Humanoid, {Name = "Emote"})
                Character.Humanoid:SetAttribute("Emoting", false)
            end
            Character.Humanoid:SetAttribute("Emoting", true)
            Animations[Animation]:Play()
```

```
            Emoting = true
            table.insert(EmoteEvents, Character.Humanoid.Jumping:Connect(function()
                StopEmote()
            end))
            table.insert(EmoteEvents, CombatMain.OnFlinch(Character.Humanoid, function()
                StopEmote()
            end))
            for a,b in pairs(EmoteEvents) do table.insert(Events, b) end
        end
        Animations = {}
        if game.ServerStorage.Characters:FindFirstChild(CharacterSelected.Value) then
            coroutine.wrap(function()
                RegisterHumanoid(Character.Humanoid)
            end)()
            local BaseChar = require(script.BaseChar):Initialize(Character)
            local CharacterScript = require(game.ServerStorage.Characters:FindFirstChild(CharacterSelected.Value))
            CharacterScript:Initialize(Character, BaseChar)
            Character.Humanoid:SetAttribute("Character", CharacterSelected.Value)
            --[[local CharacterUsing:Script = game.ServerStorage.Characters:FindFirstChild(CharacterSelected.Value):Clone()
            CharacterUsing.Parent = Character
            CharacterUsing.Enabled = true
            CharacterUsing.Name = "CharacterScript"
            CharacterUsing.Remote.OnServerEvent:Connect(function(Plr, Move, Data)
                if Player == Plr and Move == "Emote" then
                    Emote(EquippedEmotes[Data].Value)
                end
                --[[if Move == "Feint" then
                    CombatMain.Flinch(Character.Humanoid)
                elseif Move == "Ragdoll" then
                    CombatMain.Ragdoll(Character.Humanoid, 3)
                --end
            end)]]
        end
        Character.Parent = game.Workspace.Living
        LivingData = Instance.new("Folder", game.ServerStorage.LivingData)
        LivingData.Name = Player.Name
        LivingData:SetAttribute("CurrentMove", "None")
        for a,b in pairs(EquippedEmotes:GetChildren()) do
            if not Animations[b.Value] then
                Animations[b.Value] = Character.Humanoid.Animator:LoadAnimation(game.ReplicatedStorage.Animations.Emotes[b.Value])
            end
            table.insert(Events, b:GetPropertyChangedSignal("Value"):Connect(function()
                Animations[b.Value] = Character.Humanoid.Animator:LoadAnimation(game.ReplicatedStorage.Animations.Emotes[b.Value])
            end))
        end
    end)
    Respawn(Player)
end
game.Players.PlayerAdded:Connect(PlayerAdded)
for a,b in pairs(game.Players:GetPlayers()) do
    PlayerAdded(b)
end

game.Players.PlayerRemoving:Connect(function(Player)
    game.ServerStorage.LivingData[Player.Name]:Destroy()
    Data:SetAsync(Player.UserId, _G.PlayerData[Player.UserId])
    _G.PlayerData[Player.UserId] = nil
end)

game.ReplicatedStorage.Remotes.RegisterHumanoid.Event:Connect(function(Humanoid)
    RegisterHumanoid(Humanoid)
end)
```

```lua
game.ReplicatedStorage.Remotes.AddMove.OnInvoke = function(...)
    local Return = {}
    Return.Move = MoveMain.AddMoveBackend(...)
    function Return.SetEnabled(self, Visible)
        Return.Move.Visible = Visible
    end
    return Return
end

local Characters = {"Dragon of Dojima", "Dedicated Lawyer"}
game.ReplicatedStorage.Remotes.SwitchCharacter.OnServerInvoke = function(Player, Character)
    if not table.find(Characters, Character) and not AdminSystem.PlayerIsAdmin(Player) then
        AdminSystem.Ban(Player, "invalid character selection", math.huge)
        return nil
    end

    if game.Workspace:GetServerTimeNow() - Player:GetAttribute("LastSwitch") < 15 then
        return "SwitchCooldown"
    end

    if false then
        return "CombatTagged"
    end

    Player:SetAttribute("LastSwitch", game.Workspace:GetServerTimeNow())
    Player.Data.Character.Value = Character
    Respawn(Player)
    return true
end

for a,b in pairs(game.Workspace.Destroyables:GetDescendants()) do
    if b:IsA("BasePart") then
        b.CollisionGroup = "Destroyable"
    end
end

for a,b in pairs(game.Workspace.Destructables:GetDescendants()) do
    if b:IsA("BasePart") then
        b.CollisionGroup = "Destroyable"
    end
end

for a,b in pairs(game.Workspace.Ignore.ThrowableSpawns:GetChildren()) do
    local Throwable = game.ServerStorage.Throwables[b.Name]:Clone()
    Throwable.Parent = game.Workspace.Destructables.Throwables
    Throwable:PivotTo(b.CFrame)

end

game.Lighting.Atmosphere.Density = 0.3
script.GameLoaded.Value = true
```